

Component Principles

Building Microservice Systems

Ali Abrishami

Seyyed Hossein Mirhosseini



Outline

- If the SOLID principles tell us how to arrange the **bricks into walls and rooms**, then the component principles tell us how to arrange the **rooms into buildings**.
- In this lecture, we will discuss:
 - What software components are,
 - Which elements should compose them,
 - How they should be composed together into systems.

Components

- In software architecture, components are the **units of deployment**.
- They are the smallest entities that can be **deployed** as part of a system:
 - Java ➡ jar files,
 - .Net ➡ DLL files,
 - Interpreted languages ➡ aggregations of source files.

Arranging Classes in Components

- *Which classes belong in which components?*

This is an important decision, and requires guidance from good **software engineering principles**.

- Three important principles of component cohesion:
 - **REP** (Reuse/Release Equivalence Principle)
 - **CCP** (Common Closure Principle)
 - **CRP** (Common Reuse Principle)

The Reuse/Release Equivalence Principle (1)

- We are now living in the age of **software reuse**.
- A vast number of reusable components and component libraries have been created and accessed by tools such as *maven*, *pip*, and *others*.
- All components are tracked through a **release process** and are given **release numbers**.
- All the reused components are **compatible** with each other.
- The release process must produce the appropriate notifications and release **documentation** so that users can make informed decisions about when and whether to integrate the **new release**.

The Reuse/Release Equivalence Principle (2)

- This principle means that the classes and modules that are formed into a component must belong to a **cohesive** group.
- Classes and modules that are grouped together into a component should be **releasable** together.

The Common Closure Principle (1)

- **Gather** into components those classes that change for the **same reasons** and at the **same times**.
- **Separate** into different components those classes that change at **different times** and for **different reasons**.
- This is the **Single Responsibility Principle** restated for components.
- Just as the **SRP** says that a *class* should **not** contain multiples reasons to change, so the **CCP** says that a *component* should **not** have multiple reasons to change.

The Common Closure Principle (2)

- For most applications, maintainability is **more important** than reusability.
- If changes are confined to a single component, then we need to redeploy **only** the one changed component.
- Other components that don't depend on the changed component do not need to be revalidated or redeployed.
- This principle is closely associated with the **OCP** (*Open Closed Principle*).

The Common Reuse Principle (1)

- *Don't force users of a component to depend on things they don't need.*
- The Common Reuse Principle (CRP) is yet another principle that helps us to decide which classes and modules should be placed into a component.
- In such a component we would expect to see classes that have lots of dependencies on each other.

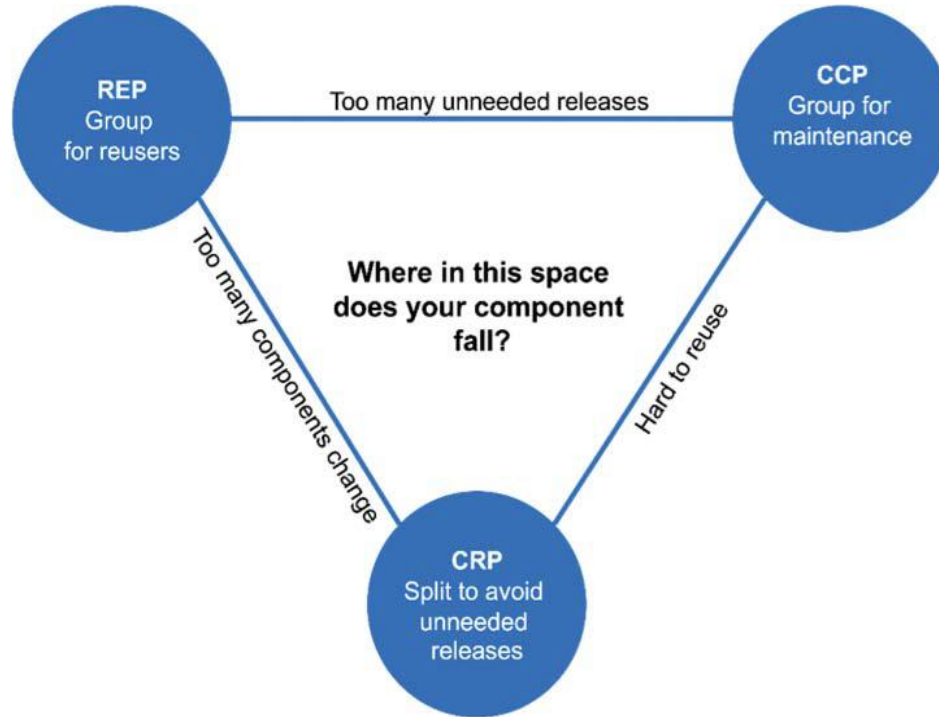
The Common Reuse Principle (2)

- It also tells us which classes **not to keep together** in a component.
- When one component uses another, a dependency is created between the components.
- The CRP says that classes that are **not** tightly bound to each other should **not** be in the same component.

CRP Relation to ISP

- The CRP is the **generic** version of the ISP.
- The ISP advises us not to depend on classes that have **methods** we don't use.
- The CRP advises us not to depend on components that have **classes** we don't use.
- ***Don't depend on things you don't need.***

Tension Diagram for Component Cohesion



Cohesion principles tension diagram

Component Cohesion Principles Tradeoffs (1)

- The REP and CCP are ***inclusive*** principles: Both tend to make components larger.
- The CRP is an ***exclusive*** principle, driving components to be smaller.
- The edges of the diagram describe the *cost* of abandoning the principle on the opposite vertex.
- A good architect finds a position in that tension triangle that meets the **current concerns** of the development team, but is also aware that those concerns **will change** over time.

Component Cohesion Principles Tradeoffs (2)

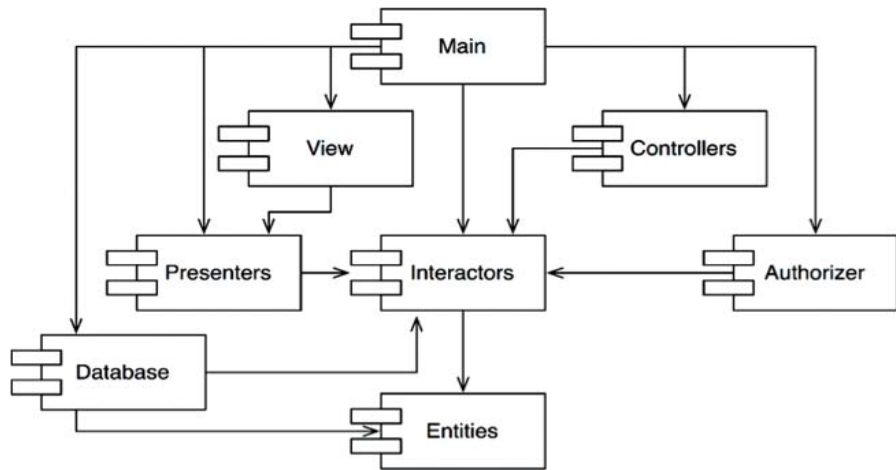
- For example, early in the development of a project, the CCP is much more important than the REP, because **developability** is more important than reuse.
- Generally, projects tend to start on the **right-hand** side of the triangle, where the only sacrifice is reuse.
- As the project matures, and other projects begin to draw from it, the project will slide over to the **left**.
- This means that the component structure of a project can vary with time and maturity.

The Acyclic Dependencies Principle

- *Allow no cycles in the component dependency graph.*
- The “morning after syndrome” occurs in development environments where many developers are modifying the same source files.
- The solution to this problem is to partition the development environment into releasable components.
- The components become units of work that can be the responsibility of a single developer, or a team of developers.
- When developers get a component working, they release it for use by the other developers.

The Acyclic Dependencies Principle (cont.)

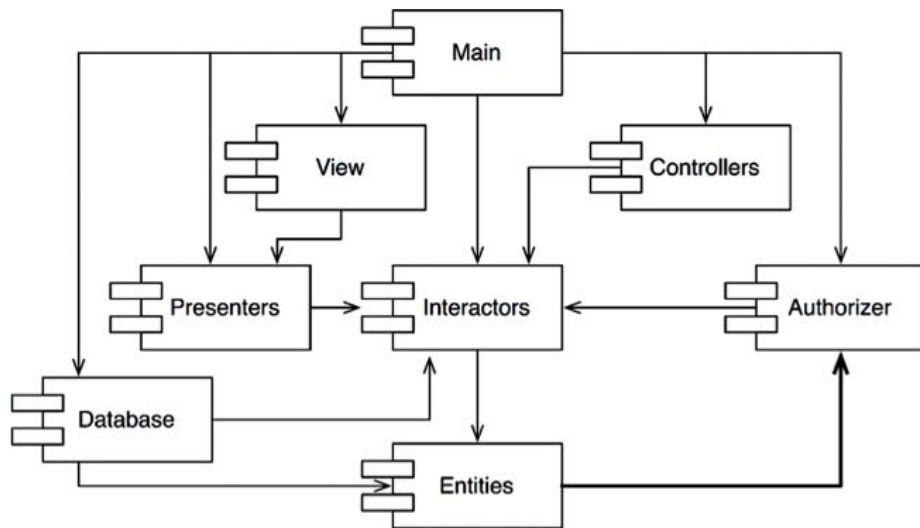
- To make it work successfully; however, you must *manage* the dependency structure of the components.
- ***There can be no cycles.***
- Example:



Typical component diagram

The Acyclic Dependencies Principle (cont.)

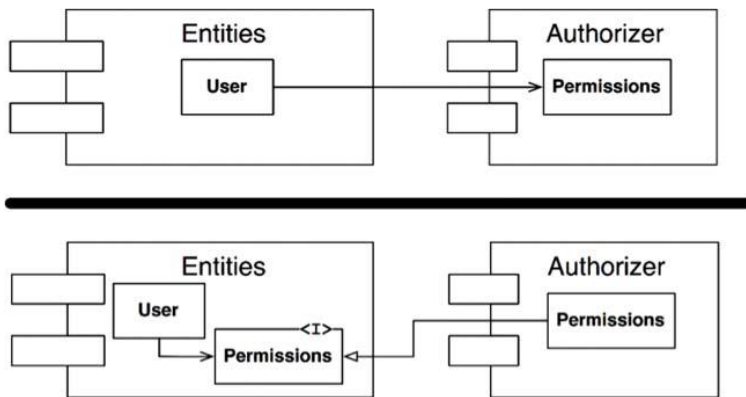
- when there are cycles in the dependency graph, it can be very difficult to work out the order in which you must build the components.



A dependency cycle

Breaking the Cycle

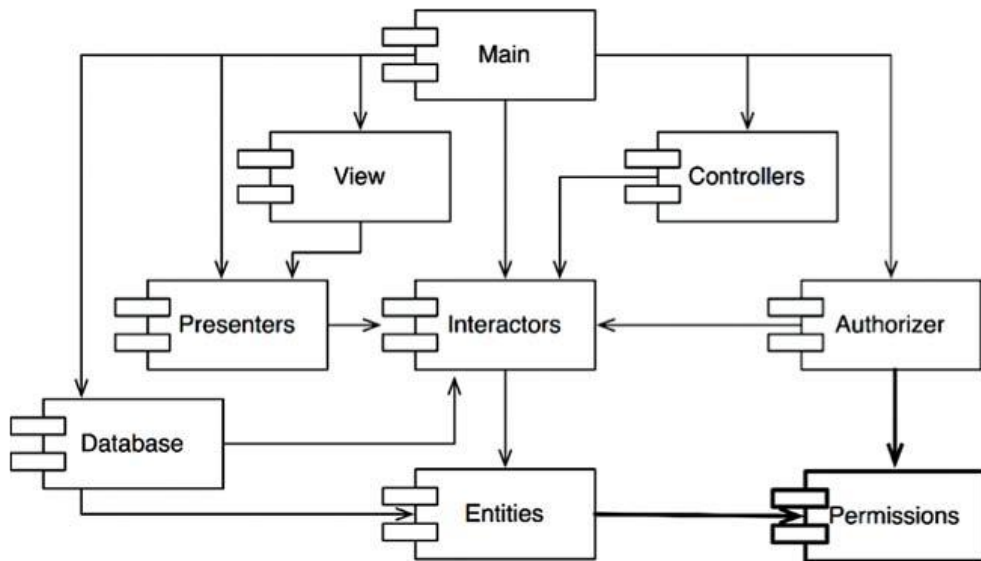
- There are two primary mechanisms for break a cycle of components and reinstate the dependency graph as a DAG:
 - 1) Apply the Dependency Inversion Principle (DIP).



Inverting the dependency between Entities and Authorizer

Breaking the Cycle (cont.)

2) Create a new component that both Entities and Authorizer depend on. Move the class(es) that they both depend on into that new component.



The new component that both Entities and Authorizer depend on

What is software architecture?

- First of all, a software architect is a **programmer**; and **continues** to be a programmer.
- Never fall for the lie that suggests that software architects pull back from code to focus on higher-level issues.
- They do not! Software architects are the **best programmers**, and they continue to take programming tasks, while they also guide the rest of the team toward a design that **maximizes productivity**.

What is software architecture? (cont.)

- Software architects may not write as much code as other programmers do, but they continue to engage in programming tasks.
- They do this because they cannot do their jobs properly if they are not experiencing the **problems that they are creating** for the rest of the programmers.
- The architecture of a software system is the shape given to that system by those who build it.
- Good architecture makes the system easy to **understand**, easy to **develop**, easy to **maintain**, and easy to **deploy**.

Development

- A software system that is hard to develop is not likely to have a long and healthy lifetime.
- A small team of five developers can quite effectively work together to develop a monolithic system without well-defined components or interfaces.
- Such a team would likely find the strictures of an architecture something of an impediment during the early days of development. This is likely the reason why so many systems lack good architecture.

Development (cont.)

- On the other hand, a system being developed by five different teams, each of which includes seven developers, cannot make progress unless the system is divided into well-defined components with reliably stable interfaces.

Deployment

- The higher the cost of deployment, the less useful the system is.
A goal of a software architecture, then, should be to make a system that can be **easily deployed** *with a single action*.
- Unfortunately, deployment strategy is seldom considered during **initial development**.
- Micro-service architecture is an example of an architecture that makes the system very easy to develop but has complex deployment process.

Operation

- Software systems that have inefficient architectures can often be made to work effectively simply by adding more storage and more servers.
- A good software architecture communicates the operational needs of the system.
- The fact that hardware is cheap and people are expensive means that architectures that impede operation are not as costly as architectures that impede development, deployment, and maintenance.

Maintenance

- Of all the aspects of a software system, **maintenance** is the **most costly**.
- Determine the best place and the best strategy to add a new feature or to repair a defect.
- A carefully thought-through architecture vastly mitigates these costs. By separating the system into components, and isolating those components through stable interfaces.

Keeping Options Open

- What are the options that we need to leave open?
*They are the **details** that don't matter.*
- All software systems can be decomposed into two major elements:
 - Policy
 - Details
- The policy element embodies all the **business rules** and **procedures**.
- The policy is where the **true value** of the system lives.

Keeping Options Open (cont.)

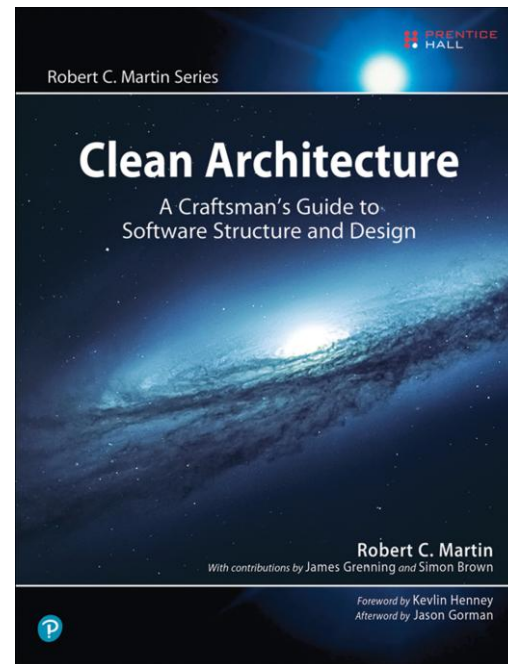
- The **details** are those things that are necessary to enable humans, other systems, and programmers to communicate with the policy, but that **do not impact the behavior of the policy** at all.
- They include IO devices, databases, web systems, servers, frameworks, communication protocols, and so forth.

Keeping Options Open (cont.)

- If you can develop the high-level policy without committing to the details that surround it, you can delay and defer decisions about those details for a long time.
- And the longer you wait to make those decisions, *the more information you have with which to make them properly.*

Resources

- Clean Architecture (Robert C. Martin): Chapters 12, 13, 14, and 15.



Q&A

Appendix

Top-Down Design

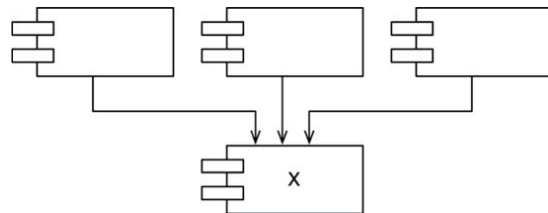
- Component dependency diagrams have very little to do with describing the function of the application.
- Instead, they are a map to the *buildability* and *maintainability* of the application.
- This is why they aren't designed at the beginning of the project.
- We start paying attention to the SRP and CCP and collocate classes that are likely to change together.
- The component dependency graph is created and molded by architects to protect stable high-value components from volatile components.

The Stable Dependencies Principle

- Some volatility is necessary if the design is to be maintained. By conforming to the Common Closure Principle (CCP), we create components that are sensitive to certain kinds of changes but immune to others.
- Some of these components are *designed* to be volatile. We *expect* them to change.
- Any component that we expect to be volatile should not be depended on by a component that is difficult to change. Otherwise, the volatile component will also be difficult to change.

Stability

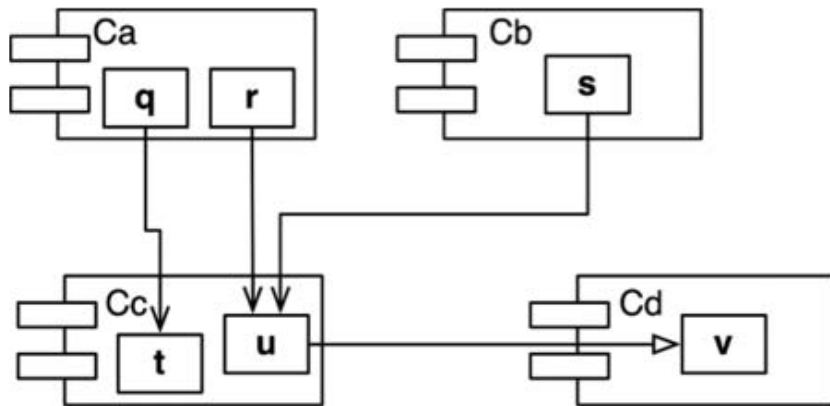
- By conforming to the Stable Dependencies Principle (SDP), we ensure that modules that are intended to be easy to change are not depended on by modules that are harder to change.
- **Stability** is related to the amount of work required to make a change.
- A component with **lots of incoming dependencies** is very stable because it requires a great deal of work to reconcile any changes with all the dependent components.



X: a stable component

Stability Metrics

- *Positional* stability of the component:
Count the number of dependencies that enter and leave that component.



Stability Metrics (cont.)

- *Fan-in*: Incoming dependencies. the number of classes outside this component that depend on classes within the component.
- *Fan-out*: Outgoing dependencies. the number of classes inside this component that depend on classes outside the component.
- *I*: Instability: $I = \text{Fan-out} / (\text{Fan-in} + \text{Fan-out})$.

This metric has the range $[0, 1]$.

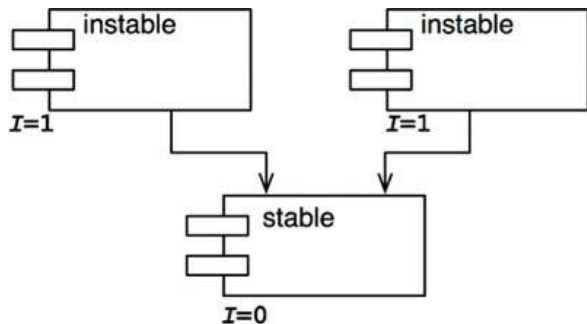
- $I = 0$ indicates a maximally stable component.
- $I = 1$ indicates a maximally unstable component.

Stability Metrics (cont.)

- In C++, these dependencies are typically represented by `#include` statements.
- In Java, the `/` metric can be calculated by counting import statements and qualified names.
- The SDP says that the `/` metric of a component should be larger than the `/` metrics of the components that it depends on. That is, `/` metrics should *decrease* in the direction of dependency.

SDP violation

- An ideal configuration for a system with three components



- Flexible is a component that we have designed to be easy to change. We want Flexible to be unstable.

